Applied NLP

DATA 641

Lecture 8 — Loopy Recurrent Neural Networks for classification

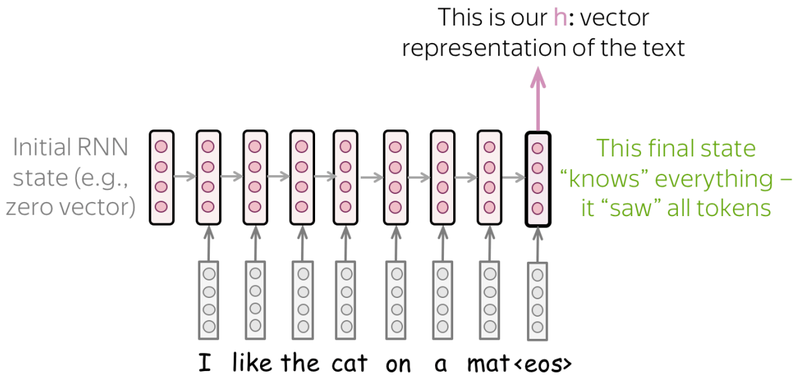
*Practical Natural Language Processing: A Comprehensive Guide to Building Real-World NLP Systems 1st Edition, Sowmya Vajjala, Bodhisattwa Majumder, Anuj Gupta, Harshit Surana, O' Reilly and Natural Language Processing in Action, Understanding, analyzing, and generating text with Python, Hobson Lane, Cole Howard, Hannes Hapke, First edition, MANNING

*Notes for this lecture are generated by the online lecture notes: https://lena-voita.github.io/nlp_course

Recurrent Neural Networks for Text Classification

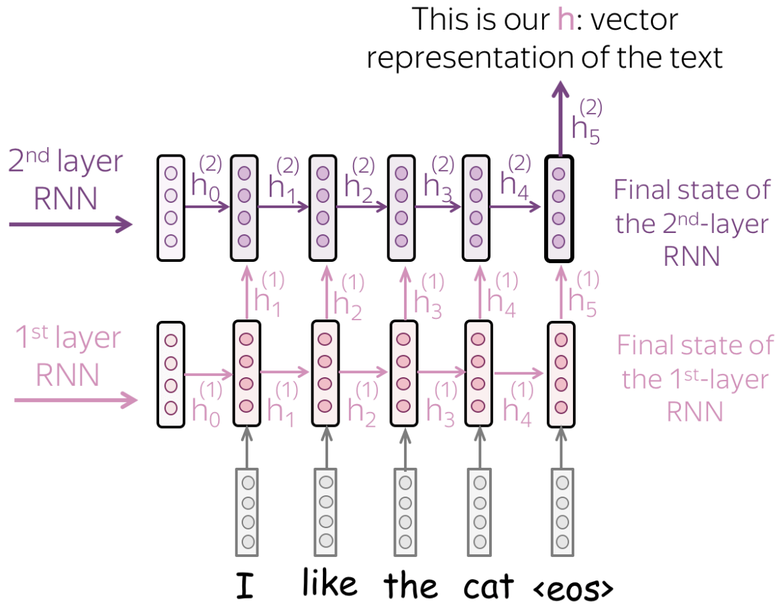
Read a text, take the final state

The most simple recurrent model is a one-layer RNN network. In this network, we have to take the state which knows more about input text. Therefore, we have to use the last state - only this state saw all input tokens.



Multiple layers: feed the states from one RNN to the next one

- To get a better text representation, you can stack multiple layers. In this case, inputs for the higher RNN are representations coming from the previous layer.
- The main hypothesis is that with several layers, lower layers will catch local phenomena (e.g., phrases), while higher layers will be able to learn more high-level things (e.g., topic).



Bidirectional: use final states from forward and backward RNNs

- Previous approaches may have a problem: the last state can easily "forget" earlier tokens. Even strong models such as LSTMs can still suffer from that!

- To avoid this, we can use two RNNs: forward, which reads input from left to right, and backward, which reads input from right to left. Then we can use the final states from both models: one will better remember the final part of a text, another - the beginning. These states can be concatenated, or summed, or something else - it's your choice!